

Trabajo Práctico

Primer semestre 2026

Introducción

En este trabajo práctico se explorarán distintos aspectos de la capa de aplicación en el modelo TCP/IP, con foco en el protocolo SMTP y su interacción con las capas subyacentes. El objetivo es construir un cliente SMTP básico implementado sobre sockets TCP para comprender en profundidad cómo viaja un correo electrónico a través de la red.

Contexto

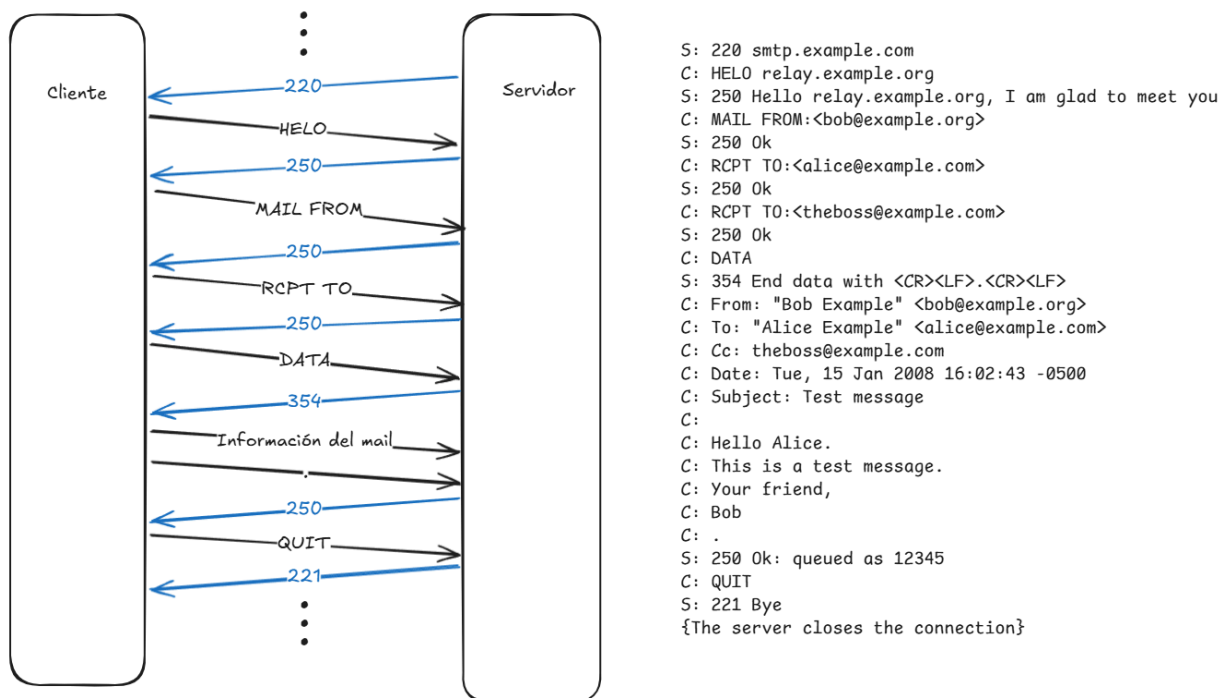
Cada día millones de mensajes son transmitidos a través del protocolo SMTP, es tan simple como elegir el destinatario, redactar el mail y enviarlo. A bajo nivel en cambio, es un proceso más complejo, primero se establece una conexión TCP con el servidor del remitente, enviando SYN, SYN+ACK y ACK. Luego se realiza el intercambio SMTP siguiendo los estándares del mismo, comandos y códigos como HELO y 220, allí es donde se envía la información propia del correo, destinatario, remitente y data. Una vez en el servidor del remitente, el mail debe ser enviado al servidor del receptor, repitiendo el procedimiento antes mencionado pero esta vez entre servidores. De esta manera, el correo queda almacenado en el servidor del receptor esperando su lectura.

En este trabajo, implementarán un cliente SMTP capaz de enviar un correo a cualquier servidor de correo, el cual deberá utilizar sockets TCP. Durante el mismo **no está permitido el uso del módulo `smtp` de Python**.

El cliente deberá funcionar de modo que pueda establecer la conexión con el servidor y manejar el intercambio de mensajes SMTP. Una vez establecida la conexión TCP si todo está correcto el servidor SMTP enviará un código de respuesta 220 junto con el nombre del servidor. A esto deberán responder con el comando HELO junto con el nombre del cliente, para identificarse ante el servidor. Al recibirlo el servidor debería enviar un código de respuesta 250 indicando que se aceptó la acción y está listo para continuar con el envío del mensaje. Estos 3 mensajes se consideran el handshake del protocolo. A continuación, se envían los comandos MAIL FROM: y RCPT TO: ambos seguidos de la información correspondiente y deberían ser contestados por el servidor con un código 250. Por último, para completar el mensaje, se envía el comando DATA, a lo que el servidor debería contestar con el código 354, lo cual indica que podemos comenzar a enviar los datos del mail, esto incluye las líneas de header (To:, From:, Cc:, Date:, Subject:) y el cuerpo del mail. Se debe enviar la información de a una línea y enviar un "." para finalizar. Si todo salió como se esperaba el servidor envía un código 250. Finalmente, se puede proceder a cerrar la conexión enviando el código QUIT, a lo que el servidor debería contestar con código 221.

Licenciatura en Tecnología Digital
TD4: Redes de Computadoras

A continuación se muestra un esquema con el intercambio de mensajes mencionado antes y datos de ejemplo.



Se proporcionará un código base que deberán completar y ampliar para cumplir con todas las consignas. El código base cuenta con la estructura mínima que deberá tener el intercambio.

Consignas

Completar el funcionamiento base del cliente:

- **Establecer conexión TCP:** Dado el servidor de mail al que elijan conectarse deberán completar el código utilizando sockets, de tal manera que establezcan una conexión con el servidor.
- **Intercambio SMTP:** Una vez establecida la conexión TCP deben completar el intercambio de mensajes SMTP con el fin de enviar un mail con el contenido elegido (remitente, destinatario y texto del mensaje). Deben respetar los estándares del protocolo, comandos y códigos (ejemplo: HELO y 220).
- **Testeo:** A la hora de evaluar la implementación deberán hacerlo usando **aiosmtpd** dado que al no incluir implementaciones de seguridad los servidores de mail reales podrían no aceptar la conexión. Deben instalar aiosmtpd con el comando **pip install aiosmtpd** en la terminal de sus computadoras. Luego, para levantar un servidor de prueba deben correr el comando **python -m smtpd -c DebuggingServer -n localhost:puerto**, reemplazando "puerto" por el número que elijan. Una vez realizado esto el servidor está listo para que se

Licenciatura en Tecnología Digital
TD4: Redes de Computadoras

conecten desde el cliente, utilizando los valores que correspondan en servidor de mail y puerto SMTP.

Análisis de protocolos e intercambio SMTP:

Ejecutar el cliente base con Wireshark activo, capturando tráfico en la interfaz de red correspondiente. Aplicar el filtro de visualización **tcp.port ==** (el puerto que corresponda). Identificar y exportar como imagen o incluir como captura de pantalla los siguientes eventos:

- 1) ¿Se puede leer el contenido del mensaje enviado? ¿En qué parte del flujo aparece? ¿Qué implicancia de seguridad tiene esto para una red Wi-Fi pública?
- 2) Buscar información de los códigos de respuesta del servidor (220, 250, 354, 221) en el RFC 5321. Para cada uno completar qué comando lo genera y su significado.
- 3) Localizar en Wireshark el segmento TCP que transporta el comando MAIL FROM. ¿Cuántos bytes tiene la carga útil (payload) de ese segmento? Comparar ese número con la longitud de la cadena enviada en el código Python.
- 4) Seleccioná el segmento TCP que contiene el comando DATA. Analizá el panel de protocolos de Wireshark y completá:
 - a) Capa de Enlace (Ethernet II):
 - Dirección MAC de origen
 - Dirección MAC de destino
 - ¿A qué equipos corresponden estas MACs? (Ayuda: ejecutar `ip neigh` en Linux o `arp -a` en Windows para ver la tabla ARP local.)
 - b) Capa de Red (IP):
 - Dirección IP origen
 - Dirección IP destino
 - ¿Cuál es tu IP privada/pública y cuál pertenece a la infraestructura del servidor?
 - c) Capa de Transporte (TCP):
 - Puerto origen:
 - Puerto destino (¿por qué este puerto?)
 - Flags activas
 - ¿Por qué están activados estos flags en este segmento específicamente?
 - d) Capa de Aplicación (SMTP):
 - Comando visualizado: DATA
 - ¿Qué significa el envío de este comando?
 - e) Si este paquete atraviesa 10 routers entre tu computadora y los servidores de Google, ¿qué información del encapsulamiento cambia en cada salto y qué se mantiene constante? Explicar por qué.

Guardar la captura de Wireshark de esta sesión sin cifrar (File > Save As, formato .pcapng). Se va a comparar directamente con la captura cifrada que se hará después de implementar STARTTLS en la Parte II.

Implementación de seguridad:

Una vez completado el cliente base, deberán extender la implementación para soportar comunicaciones seguras. Para esto, partiendo del código provisto en el archivo *smtpCliente_TLS.py*, deberán incorporar el comando STARTTLS que permite elevar una conexión TCP existente a un canal cifrado mediante TLS, y la autenticación AUTH LOGIN que permite identificarse ante el servidor utilizando usuario y contraseña codificados en Base64.

A diferencia de la parte anterior, esta versión sí podrá conectarse a un servidor de mail real. Se recomienda utilizar Gmail (smtp.gmail.com, puerto 587).

- **STARTTLS:** Luego del EHLO inicial, enviar el comando STARTTLS y esperar la respuesta 220 del servidor. Una vez confirmada, envolver el socket con `ssl.SSLContext` y `wrap_socket()` para cifrar el canal. Tras esto, volver a enviar EHLO sobre el canal cifrado antes de continuar.
- **AUTH LOGIN:** Enviar el comando AUTH LOGIN y responder a los dos desafíos del servidor (código 334) con el usuario y la contraseña codificados en Base64 usando el módulo `base64`. Verificar que el servidor responda con el código 235 antes de continuar con el envío.
- **Credenciales: No hardcodear usuario ni contraseña en el código.** Deben gestionarse mediante variables de entorno (SMTP_USER y SMTP_PASS). Para Gmail en particular, no se acepta la contraseña de la cuenta sino una **App Password** (contraseña de aplicación), que puede generarse de la siguiente manera:

1. Conseguir la App Password de Gmail

Entrá a: myaccount.google.com/apppasswords

Creá una con cualquier nombre (ej: "TP Redes"), copió las 16 letras que te da.

2. Configurar las variables de entorno en la terminal

```
$env:SMTP_USER = "tu_cuenta@gmail.com"
```

```
$env:SMTP_PASS = "xxxx xxxx xxxx xxxx"
```

Testeo con Wireshark: Repetir la captura de tráfico de la sección anterior pero ahora sobre la interfaz Wi-Fi con el filtro **tcp.port == 587**. Identificar el TLS Client Hello, el Server Hello y el momento a partir del cual el tráfico aparece como Application Data y deja de ser legible, comparando con la captura de texto plano de la parte anterior.

Experimentación:

Realizar una serie de pruebas comparativas usando tanto la implementación básica como la versión con seguridad del cliente SMTP. El objetivo es analizar cómo cambia la aceptación del envío de correos según el mecanismo de seguridad y el tipo de servidor o destinatario utilizado.

Para ello, deberán probar distintas combinaciones posibles, por ejemplo:

Licenciatura en Tecnología Digital
TD4: Redes de Computadoras

- servidor local de prueba
- cuentas de Gmail
- servicios de correo temporal

En cada caso, se debe registrar:

- si la conexión fue aceptada o rechazada
- si el envío del correo pudo completarse o no
- si fue necesaria autenticación
- si la autenticación resultó exitosa o fallida
- el código SMTP final recibido

A partir de los resultados obtenidos, se debe comparar el comportamiento de ambas implementaciones y discutir cuando el uso de mecanismos de seguridad influye en la posibilidad de enviar correctamente un correo.

Informe

Deberán realizar un informe con **máximo 6 páginas** (sin contar la carátula) que documente las decisiones relevantes tomadas durante la implementación, los errores encontrados y, en caso de haberlos solucionado, el proceso seguido para hacerlo. Además, deberán registrar de forma detallada los puntos de análisis y experimentación solicitados en la consigna.

El informe deberá tener como mínimo los siguientes apartados:

Introducción:

Breve descripción del objetivo del trabajo práctico, el contexto del protocolo SMTP y la finalidad del código implementado.

Implementación:

Descripción técnica de las decisiones de diseño y desarrollo tomadas para cumplir con las consignas. Diferencias entre la implementación base y la implementación con seguridad. También se deben mencionar los problemas encontrados y las soluciones aplicadas.

Experimentación:

Registro de las pruebas realizadas, comparativa entre la implementación base y la versión con seguridad. Se espera que presenten los resultados de forma clara, acompañados de tablas y/o gráficos comparativos. También deben incluir una breve interpretación de los resultados obtenidos.

Conclusión:

Resumen final del trabajo, destacando los aprendizajes obtenidos, los principales desafíos técnicos enfrentados y posibles mejoras futuras. Se espera que el grupo explique qué se buscó aprender o demostrar con el desarrollo del cliente smtp, y cómo se relaciona con los temas vistos en clase.

Se alienta a incluir también cualquier punto adicional de experimentación, análisis o implementación que consideren pertinente, incluso si no fue requerido explícitamente.

Modalidad de entrega y evaluación

El trabajo debe realizarse en **grupos de 3 alumnos**. Deberán entregar **un solo archivo .zip con sus apellidos** como nombre del mismo que contenga:

- Ambos códigos fuentes (**.py**) debidamente comentados.
- El informe en formato PDF.

La entrega debe realizarse a través del campus virtual, con fecha límite el 15 de junio de 2026 a las 14:00 hs.

No se aceptarán trabajos entregados fuera de término.

El trabajo será evaluado de la siguiente manera: **Código x 0,4 + Informe x 0,6**. Donde el código debe correr sin problemas y realizar lo pedido, y el informe debe mostrar lo trabajado, con cualquier problema que pueda haber surgido. Adicionalmente, se realizará una **defensa oral obligatoria de 15 minutos** por grupo, la fecha se confirmará en cada sección. Esta defensa tiene como objetivo constatar el trabajo realizado y, según el desempeño durante la misma, podrá impactar en la nota final del trabajo práctico.

Documentación

Les dejamos una guía de documentos donde podrán conseguir información para realizar las implementaciones del cliente, no es necesario usarlas o hacerlo exactamente como dicen estos documentos, es simplemente a modo de ayuda para que tengan fuentes donde informarse.

En caso de no tener **Wireshark** o **pip install** pueden revisar la guía que se encuentra en el archivo de "Herramientas" en el campus virtual.

Socket Programming in Python:

<https://realpython.com/python-sockets/>

<https://www.geeksforgeeks.org/python/socket-programming-python/>

<https://docs.python.org/3/library/socket.html#socket-objects>

Códigos y comandos SMTP:

https://en.wikipedia.org/wiki/List_of_SMTP_server_return_codes

<https://www.geeksforgeeks.org/computer-networks/smtp-commands/>

Licenciatura en Tecnología Digital
TD4: Redes de Computadoras

SMTP AUTH:

https://en.wikipedia.org/wiki/SMTP_Authentication

STARTTLS:

https://en.wikipedia.org/wiki/Opportunistic_TLS

<https://docs.python.org/3/library/ssl.html>